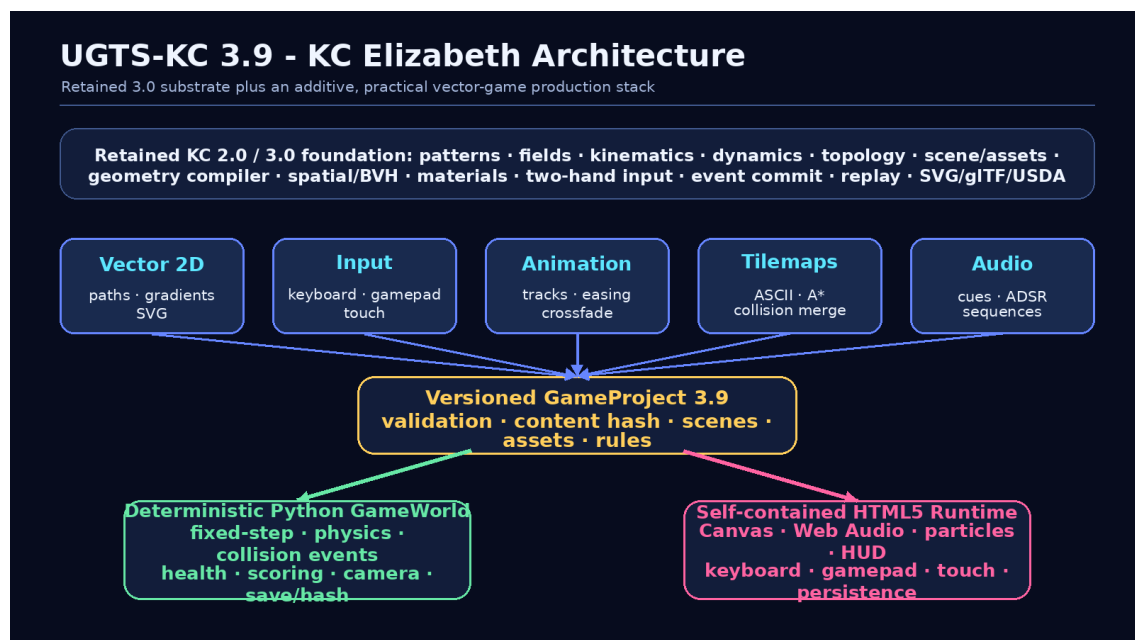


UGTS-KC 3.9

KC ELIZABETH EDITION

Vector Art, Deterministic Game Runtime and Self-Contained HTML5 Export



Release architecture overview

Prepared for Tom Klootwijk
Requester-supplied identifier: NL200678942
Requester-supplied date of birth: 10-07-1990
Substrate attribution as supplied: Kees Klootwijk (KC edition)

Version 3.9.0 | Project schema [ugts-kc-game-project-3.9](#)

Contents and Executive Summary

Section	Purpose
1. Release definition	What 3.9 adds and what remains compatible.
2. Architecture	How vector assets, project data, simulation and browser output connect.
3. Authoring systems	Vector, input, animation, tilemap and audio APIs.
4. Runtime systems	Entity/components, physics, collisions, gameplay, camera and persistence.
5. Browser deployment	Self-contained Canvas/Web Audio game output and controls.
6. Demonstration	Elizabeth's Vector Garden vertical slice.
7. Validation	Tests, catalogs, schema, syntax, builds and captured evidence.
8. Compatibility and boundary	Migration from 3.0, explicit non-claims and roadmap.

Executive summary

KC Elizabeth 3.9 upgrades the supplied KC Two Hands 3.0 archive from a strong graphics/interaction reference layer into an end-to-end 2D vector-game creation stack. The original 3.0 APIs and 117 tests remain present. The new layer adds a project model, vector artwork, multi-device input, collision/physics, animation, tilemaps/pathfinding, procedural audio, deterministic game simulation, command-line tooling and an offline browser build target.

Release result

A single editable project record can be validated and simulated in Python, then built into a self-contained browser game with keyboard, gamepad and touch controls - without external runtime libraries or network assets.

Measured expansion

- 60 new cataloged mechanisms, M330-M389, extending the combined catalog to 389 mechanisms.
- 108 new automated tests, bringing the complete retained-plus-new suite to 225 tests.
- 11 additive Python modules for vector art, collision, input, animation, tilemaps, audio, game world, projects, HTML5 output, templates and CLI/version metadata.
- A complete demonstration project with editable JSON, exported SVG source assets, deterministic headless smoke run and offline browser build.

1. Release Definition and Scope

The release preserves the canonical UGTS authority chain and wraps it in a pragmatic game-production loop. The game layer is additive: it does not replace the retained scene, geometry compiler, hand interaction, event proposal/commit, replay or interchange modules.

```
local/spatial support -> compatibility -> guard classification
-> verified proposal -> deterministic event commit
-> scene/topology/dynamic patch -> lineage + replay log
-> optional render/export

vector assets + input actions + scene project
-> fixed-step game world + collision/gameplay systems
-> canonical snapshot/hash/save
-> Canvas/Web Audio HTML5 build
```

Area	KC Two Hands 3.0 baseline	KC Elizabeth 3.9 addition
Geometry/content	Scene, assets, geometry compilation, materials, SVG/glTF/USDA	Serializable 2D paths, gradients, primitive factories and per-asset SVG.
Interaction	Typed two-hand input and bimanual transforms	Action map spanning keyboard, pointer, gamepad and touch.
Runtime	Deterministic proposal commit, snapshots and replay	Fixed-step 2D game world, body integration, collisions and gameplay components.
Authoring	Python APIs and sandbox	Versioned project JSON, validation, templates and CLI.
Delivery	Reference render/export adapters	Self-contained playable HTML5 Canvas/Web Audio output.
Coverage	117 tests, catalog through M329	225 tests total, catalog through M389.

Compatibility guarantee

All retained source modules remain available under their previous paths. The upgrade is additive under `ugts_kc3`. The game-project schema is explicitly versioned, and legacy reports/evidence are preserved in dedicated legacy folders.

Pragmatic design choices

- Dependency-free engine runtime: standard-library Python and generated browser APIs.
- Small, inspectable records rather than opaque binary assets.
- Deterministic fixed-step reference simulation for tests and save identity.
- A browser target that can be opened directly or served by any static host.
- Extensible custom component mappings and ordered system phases for project-specific logic.

2. Architecture

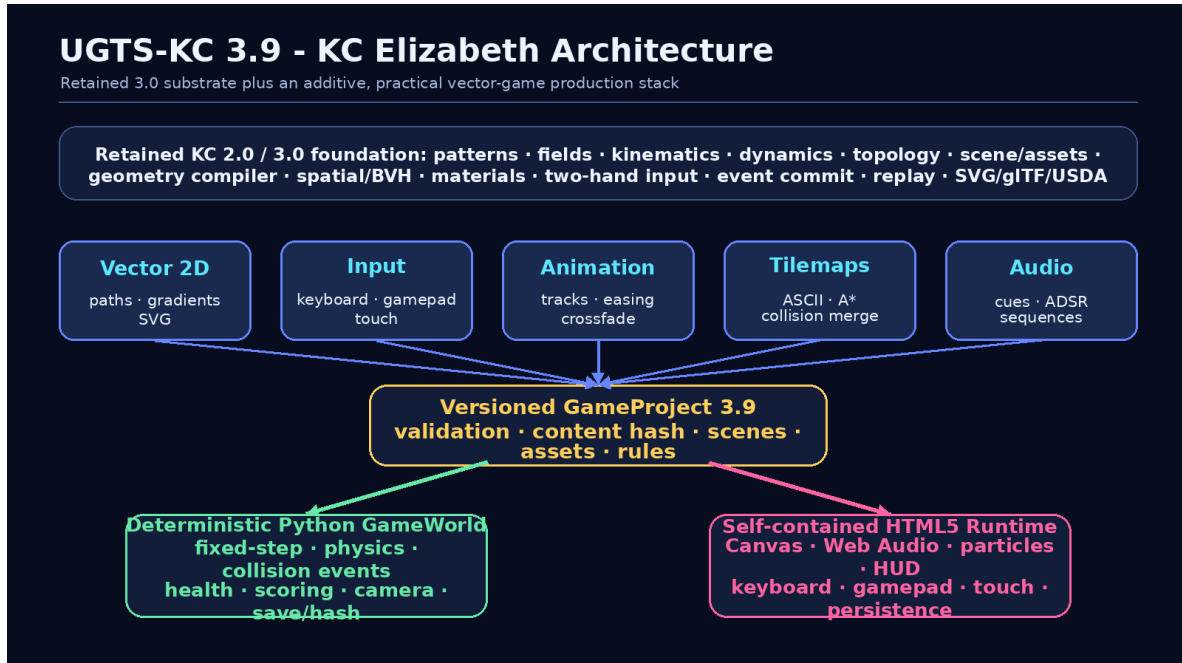


Figure 1. Retained substrate and additive 3.9 game stack.

Single source of project truth

GameProject holds metadata, display settings, input actions, vector assets, audio records, tilemaps, scenes, the start-scene ID and build settings. The same canonical dictionary is used for project validation, content hashing, Python scene instantiation and browser embedding.

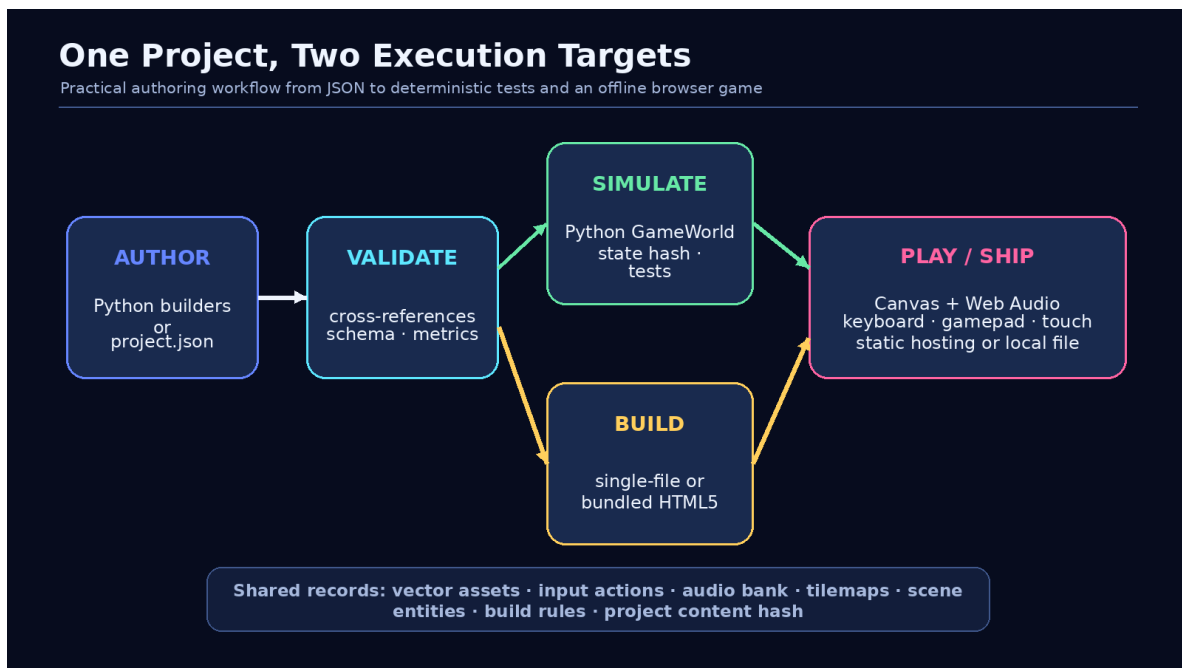


Figure 2. One project feeds headless simulation and browser deployment.

3. Vector Art Authoring

The new vector layer supplies a compact retained-mode artwork format suitable for code generation, JSON storage, SVG inspection and direct Canvas rendering. Assets remain resolution independent and avoid external texture files for the included workflow.

Record	Role	Key guards
VectorAsset2D	Stable asset ID, logical size, pivot, ordered paths, gradients and metadata.	Positive size, unique path/gradient IDs, valid references.
VectorPath	Typed command sequence plus paint and fill rule.	Move-before-draw, finite coordinates, supported commands.
VectorPaint	Fill, stroke, width, opacity, cap/join and fill rule.	Opacity range and non-negative stroke width.
Linear/RadialGradient	Named reusable stop resources.	Finite geometry, positive radius and ordered offsets.
VectorLibrary	Deterministic asset registry.	Duplicate IDs rejected unless explicitly replaced.

Curve and primitive support

- Path commands: move, line, quadratic, cubic and close.
- Adaptive quadratic/cubic flattening to a caller-selected tolerance.
- Rectangle/rounded rectangle, circle, polygon and star factories.
- Deterministic SVG output with correctly centered size/pivot framing.

```
path = (VectorPathBuilder("ship")
    .move_to(28, 0).line_to(-20, -18)
    .line_to(-8, 0).line_to(-20, 18)
    .close().build(VectorPaint(fill="@hull", stroke="#ffffff")))
asset = VectorAsset2D("ship", (64, 48), (0, 0), (path,), gradients)
```

Regression found during packaging

Standalone SVG inspection exposed an incorrect viewBox origin for centered procedural assets. Version 3.9 now frames the logical rectangle around the declared pivot, and the test suite includes a regression assertion.

4. Input and Animation

Action-based multi-device input

Gameplay logic consumes named scalar actions. Bindings can sample keys, pointer controls, gamepad axes/buttons and touch axes. Action resolution applies signed scaling, deadzones, optional clamping and thresholds. InputFrame exposes value, down, pressed, released and normalized two-action vectors.

Action pattern	Example bindings	Use
move_x	A/D, arrows, gamepad axis 0, touch move_x	Horizontal movement.
move_y	W/S, arrows, gamepad axis 1, touch move_y	Vertical movement.
dash	Space, Shift, gamepad button 0, touch dash	Edge-triggered burst movement.
pause/restart/save/load	Keyboard and gamepad buttons	Game-shell control.
debug/mute	F3 and M	Runtime diagnostics and audio control.

Deterministic input frames

InputMap resolves raw device state or explicit action values into serializable frames containing current and previous values, thresholds, pointer position and frame index. InputRecorder stores and restores frame sequences for tests and replay-oriented tooling.

Keyframe animation

- Strictly increasing key times and finite scalar/vector values.
- Named easing functions including linear, smooth, polynomial, sine, back and bounce families.
- Once, loop and ping-pong clip time mapping.
- Crossfade blending and predicate-driven animation state machines.
- Dot-path sample application into nested serializable records.

```
clip = AnimationClip("pulse", 1.0, tracks, mode="pingpong")
player = AnimationPlayer({"pulse": clip})
player.play("pulse")
sample = player.update(1 / 60)
apply_animation_sample(component_record, sample)
```

5. Tilemaps, Navigation and Procedural Audio

Layered tilemaps

TileMap stores named definitions and same-sized layers over a common origin and tile size. Compact ASCII import makes prototype levels readable in source control. The map API handles tile/world conversion, solidity queries, flood reachability, cost-aware A* paths and merged solid collision rectangles.

Operation	Contract	Practical use
ASCII import	Rectangular rows and complete character legend.	Fast editable prototype maps.
A* path	In-bounds non-solid endpoints; optional diagonals and movement cost.	Grid enemies and puzzle navigation.
Flood search	Returns all reachable cells from a start.	Room/region and placement analysis.
Collision merge	Adjacent compatible solid cells become larger AABBs.	Fewer world colliders.

Procedural sound records

AudioBank validates named oscillator cues and beat sequences. SoundCue combines waveform, start/end frequency, duration, volume, optional noise and an ADSR envelope. Notes use equal-tempered pitch conversion. Python validates and serializes these records; the generated browser runtime realizes them through Web Audio.

- No sampled audio files are needed for the included effects/music.
- Cue references are validated across entities, scene rules and sequences.
- The demonstration includes collect, dash, damage, win, game-over, save and music-pluck cues.

Runtime boundary

Procedural audio playback is implemented in the browser target. The Python layer intentionally remains device-independent data/validation rather than opening an audio device.

6. Deterministic Game World



Figure 3. Authoritative fixed-step system order.

GameWorld is a dependency-free entity/component reference runtime. Every simulation tick runs an explicit phase order. Custom systems register by phase, integer priority and stable name, so projects can extend gameplay without weakening deterministic ordering.

Component	Function
Transform2D	Position, rotation and non-zero scale.
Body2D	Dynamic/kinematic/static body, velocity, force, damping, gravity scale and material response.
Collider2D	AABB/circle/convex shape, offset, layer/mask filter and sensor flag.
PlayerController2D	Action-driven velocity and edge-triggered dash with duration/cooldown.
Health2D / Hazard2D	Bounded health, invulnerability, damage cooldown and knockback.
Collectible2D	State-key scoring, sound event and optional deferred destruction.
BoundsConstraint2D	Clamp, bounce or wrap world constraints.
Camera2D / Lifetime2D	Smoothed bounded follow camera and timed entity removal.

7. Collision, Gameplay Events and Persistence

Collision stack

- Shape support: AABB, circle and convex polygon.
- Mixed-shape manifold dispatch with penetration, normal and contact points.
- Layer/mask filtering and non-resolving sensor contacts.
- Spatial-hash broad phase for deterministic candidate pairs.
- Swept AABB time-of-impact query for fast box motion.
- Velocity response with restitution and friction plus penetration correction.

Lifecycle and gameplay events

The contact set is compared across ticks to emit `collision_enter`, `collision_stay` and `collision_exit`. Collectibles and hazards are built on these contacts. Entity destruction requested during a step is deferred until the tick completes, preventing mutation during collision iteration.

Event example	Payload/use
<code>entity_spawned / entity_despawned</code>	Lifecycle inspection and custom systems.
<code>collision_enter/stay/exit</code>	Ordered entity pair, normal, penetration and contacts.
<code>dash</code>	Source entity and dash direction.
<code>collected</code>	Collector, points and state key.
<code>damaged / entity_defeated</code>	Damage, remaining health and source hazard.
<code>sound</code>	Cue ID consumed by presentation or test harness.

Snapshots, hashes and saves

Snapshots include schema, fixed step, gravity, tick/time, game state, entity/component records, active contacts and optional event history. State hashes intentionally exclude event history and use canonical sorted JSON. Numerically equivalent integral spellings, such as 1 and 1.0, normalize to the same identity, preventing hash drift after JSON round trips.

```
snapshot = world.snapshot()
world.save("save.json")
clone = GameWorld.load("save.json")
assert clone.state_hash() == world.state_hash()
```

8. Self-Contained HTML5 Runtime

build_html5 validates a GameProject and emits either one self-contained HTML file or an HTML page plus kc-runtime.js. The build also writes the canonical project JSON, launch notes and a build report with project/file hashes and byte counts.

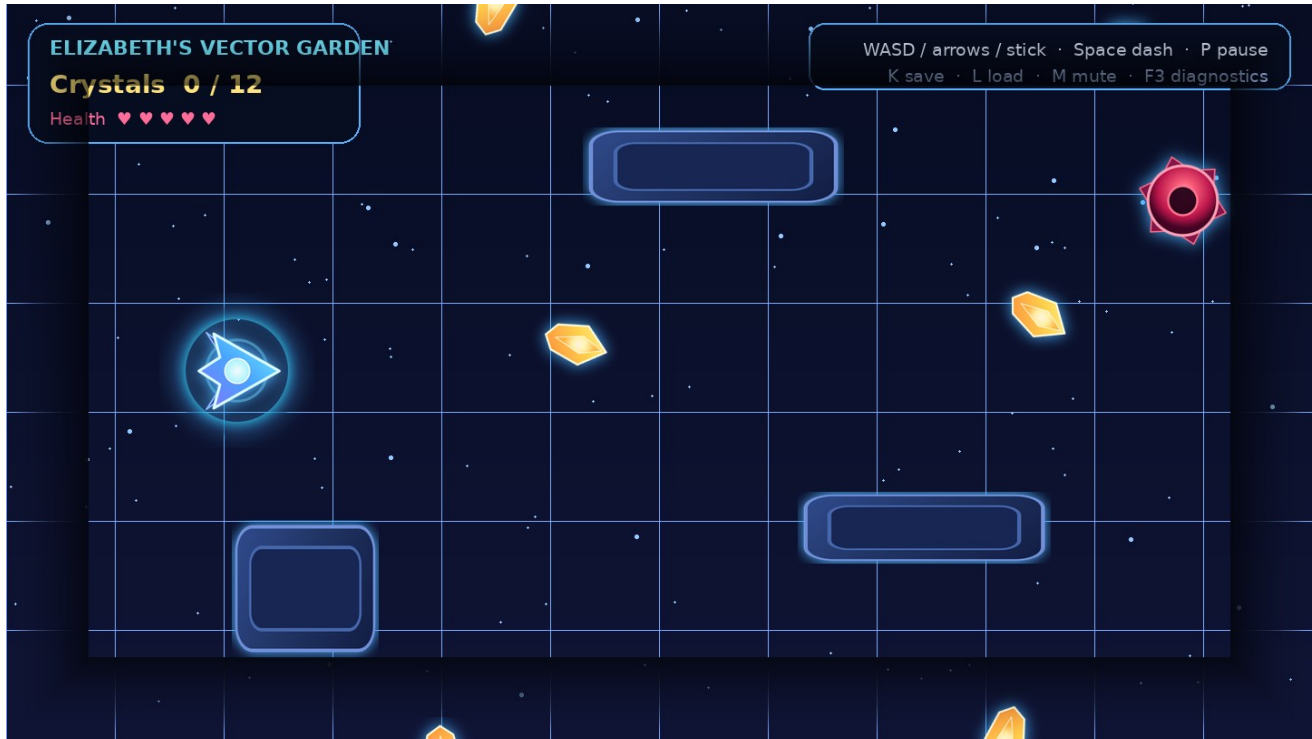


Figure 4. Static inspection render generated from the same vector project records used by the browser build.

Browser subsystem	Implemented behavior
Presentation	Canvas paths, fills/strokes, linear/radial gradients, z-order, transforms, camera, grid and vignette.
Simulation	Fixed-step accumulator, body integration, broad/narrow collision, bounds, pickups, hazards and win/lose state.
Input	Keyboard, gamepad polling and touch joystick/dash controls.
Feedback	Particles, HUD, best score, pause/restart, mute and F3 diagnostics.
Audio	Web Audio oscillator/noise cues and beat-sequence music.
Persistence	Project-ID-scoped local save/load and best-score storage.

Deployment

The single-file output uses no CDN, package manager or network asset. It may be opened directly or hosted by any static server. Browser storage is convenience persistence, not a security boundary.

9. Command-Line Authoring Workflow

The module entry point and installed `ugts-kc` command make the package usable without writing a custom build script.

Command	Purpose
<code>info</code>	Print runtime version, edition and capabilities.
<code>new</code>	Create a blank or Elizabeth-quest project; optionally build immediately.
<code>validate</code>	Aggregate project errors/warnings and metrics; optional JSON report.
<code>simulate</code>	Run deterministic headless steps with declared movement/dash values.
<code>build-web</code>	Create single-file or split-bundle browser output.
<code>export-svg</code>	Write every vector asset as an individual SVG.
<code>demo</code>	Write and build Elizabeth's Vector Garden.

```
PYTHONPATH=src python -m ugts_kc3 new my_game \
  --title "My Vector Game" --author "Your Name" --build

PYTHONPATH=src python -m ugts_kc3 validate my_game/project.json
PYTHONPATH=src python -m ugts_kc3 simulate my_game/project.json \
  --steps 240 --move-x 1 --dash-at 30 --json

PYTHONPATH=src python -m ugts_kc3 build-web \
  my_game/project.json my_game/dist
```

Project validation metrics

- Scene and entity counts.
- Vector asset, audio cue, tilemap and input-action counts.
- Metadata/display/input/audio validation.
- Duplicate IDs and start-scene checks.
- Unknown vector asset, sound cue and tilemap references with exact paths.

10. Elizabeth's Vector Garden Demonstration

The included vertical slice is not merely a screenshot or design document. It is an editable project.json, a set of exported SVG assets, a deterministic headless smoke script and a generated offline browser game under [examples/elizabeth_vector_quest/](#).

Content	Demonstrated capability
Player ship	Vector gradients, camera follow, collision body, action movement, dash, health and bounds.
12 crystals	Sensor collision, score transaction, particles, sound, spin/pulse and win target.
7 obstacles	Static bodies, scaled vector assets and collision response.
4 moving hazards	Kinematic patrol, bounce bounds, damage cooldown, knockback and game-over path.
Portal/background	Decorative vector layers, grid and vignette.
HUD/shell	Instructions, score/health/best state, pause, restart, saves, mute and diagnostics.
Audio	Procedural effects plus an eight-beat music sequence.

Controls

- Move: WASD, arrow keys, gamepad left stick or touch joystick.
- Dash: Space, Left Shift, gamepad button 0 or touch dash.
- Pause: P or Escape. Restart: R. Save: K. Load: L. Mute: M. Diagnostics: F3.

Headless smoke evidence

The checked-in headless_smoke.py loads the exact project file, runs 240 fixed steps with a deterministic movement/dash input sequence and prints tick, state, entity count and state hash. This exercises project loading, scene instantiation, input frames, game stepping and canonical state identity without a browser.

```
{"tick": 240, "state": {"score": 1, "scene": "vector_garden"},
  "entities": 25,
  "hash": "cf1317428c6992255e8911f3647a3faaa959f4034b7ccec8ebc92c7b6f4db8ae"}
```

11. Validation and Release Evidence

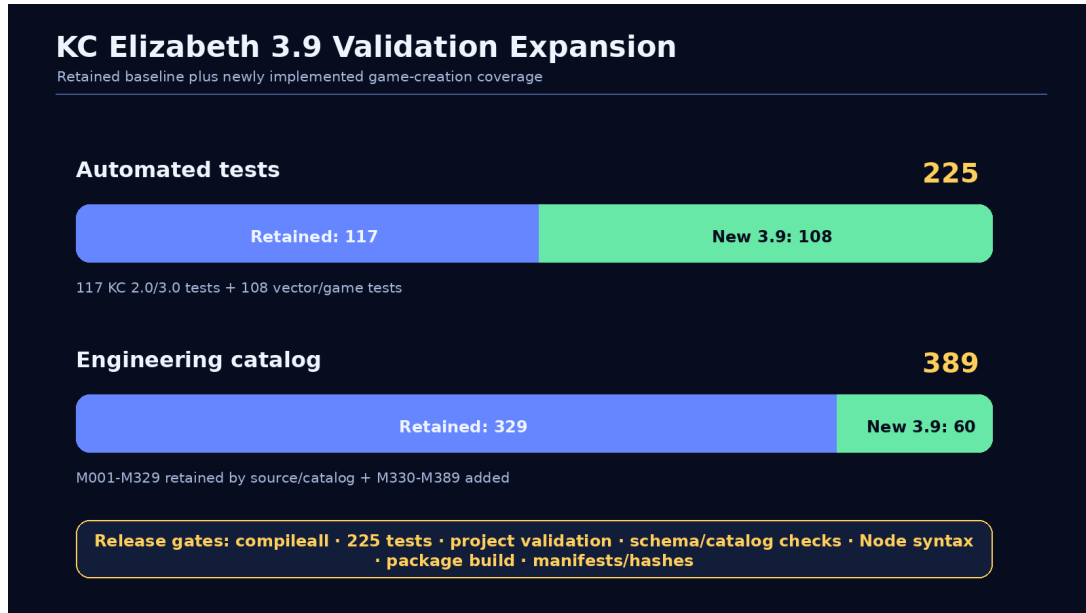


Figure 5. Test and mechanism-catalog expansion.

Gate	Expected release result
Python compile	All source modules compile with compileall.
Automated tests	225/225 pass: 117 retained and 108 new.
Project validation	Blank and Elizabeth templates pass cross-reference validation.
Round trips	Project and world save/load preserve canonical content/state hashes.
Catalog	M198-M389 engineering catalog is contiguous; M330-M389 contains 60 additions.
JSON schema	Example/template projects satisfy top-level 3.9 schema and runtime validation.
JavaScript	Generated runtime passes Node.js syntax validation.
Browser build	Single-file and split-bundle outputs are generated with reports/ hashes.
Package build	Python wheel/sdist metadata build succeeds.
Manifest	All final files receive SHA-256 entries and release metrics.

The validation directory contains machine-readable summaries, command captures, catalog/schema checks, build reports, file manifests and hashes. The original 3.0 evidence is preserved under validation/legacy_3_0 rather than overwritten.

12. Mechanism Catalog M330-M389

KC Elizabeth adds 60 engineering mechanisms across eight domains. Full definitions, formulas, integration notes and validation statuses are available in CSV and JSON under spec/.

Range	Domain	Count	Representative mechanisms
M330-M337	Vector art	8	Asset records, path grammar, gradients, curve flattening, primitives and SVG.
M338-M345	Collision/physics	8	Shapes, SAT, mixed manifolds, filters, sensors, sweep and spatial hash.
M346-M351	Input	6	Actions, multi-device bindings, deadzones, edges, vectors and recording.
M352-M357	Animation	6	Tracks, easing, clip clocks, crossfades, state machines and property paths.
M358-M363	Tilemaps	6	Definitions, layers, ASCII import, transforms, A* and merged collisions.
M364-M368	Procedural audio	5	Pitch, ADSR, cues, sequences and cross-reference bank.
M369-M378	Game world	10	ECS, fixed step, phases, bodies, contacts, controller, gameplay, saves.
M379-M389	Project/export	11	Schema, validation, instantiation, hashes, CLI, simulation, SVG and HTML5.

Combined count

```

Source-normalized baseline: 197 mechanisms
KC 2.0 additions:          60  (M198-M257)
KC 3.0 additions:          72  (M258-M329)
KC Elizabeth 3.9:          60  (M330-M389)
-----
Extended total:             389 mechanisms

```

Catalog meaning

Mechanism IDs are engineering catalog entries for traceability and coverage. They are not, by themselves, claims of novelty, patentability, ownership or independent scientific validation.

13. Migration, Compatibility and Evidence Boundary

Migration from 3.0

1. Retain existing KC 3.0 scene/geometry/hand/replay workflows where they are useful.
2. Create a VectorLibrary for browser-ready 2D art.
3. Define device-independent actions in InputMap.
4. Express playable objects as EntitySpec component records.
5. Create GameSceneSpec rules and initial state, then wrap them in GameProject.
6. Validate, simulate headlessly and build HTML5 output.
7. Use GameWorld snapshots/hashes for 2D game tests and retained 3.0 replay/event facilities for higher-level authoring workflows.

Explicit non-claims

- No native Vulkan, Direct3D, Metal or physical-device WebGPU implementation is claimed.
- No general rigid-body/joint/deformable physics engine is claimed.
- No skeletal animation suite, multiplayer service, console SDK or OpenXR binding is claimed.
- No full OpenUSD, MaterialX or OCIO runtime integration is claimed.
- No security certification is claimed for project files or browser storage.
- Requester identity, attribution and ownership assertions are recorded as supplied and remain independently unverified.

Post-3.9 direction

The roadmap prioritizes an optional Rust/C ABI core, visual editor, richer asset support, continuous collision and joints, browser tilemap rendering/streaming, optional WebGPU presentation, import adapters, multiplayer proof work and cross-browser automation. Each remains future work rather than an implicit feature claim.

Appendix A - Package Map and Attribution

Path	Contents
src/ugts_kc/	Retained KC 2.0 pattern/topology/kinematics/field/dynamics oracle.
src/ugts_kc3/	Retained 3.0 modules plus 3.9 vector/game/project/browser stack.
examples/elizabeth_vector_quest/	Editable project, SVG assets, screenshot, smoke script and offline build.
templates/	Blank and full project templates.
spec/	Contracts, JSON schema and catalogs through M389.
docs/	Getting started, creation guide, API overview, migration, runtime guide and boundaries.
report/	Current 3.9 report and retained 3.0 legacy reports.
tests/	225 automated tests.
validation/	Current release evidence and retained legacy evidence.

Attribution notice

Prepared for Tom Klootwijk as the UGTS-KC 3.9 - KC Elizabeth Edition. The requester described the underlying substrate attribution as Kees Klootwijk (KC edition). The identifier NL200678942 and date of birth 1990-07-10 (request format 10-07-1990) were supplied by the requester and are not independently verified.

This report and package are not legal proof of identity, authorship, ownership, patentability, priority, exclusive rights, licensing status or chain of title. No third-party source is silently re-licensed by the attribution wording.

Release identifiers

Identifier	Value
Runtime version	3.9.0
Codename	KC Elizabeth
Project schema	ugts-kc-game-project-3.9
Combined mechanism range	M001-M389 by source/catalog
Automated tests	225
Immediate baseline SHA-256	624f9af1236ea6a0a637758516ed974238ad92b324949f690f0fcc169d9f7866